A project report on

SMARTPACK: CLOSURE-VERIFIED RFID SMART BAG WITH TOGGLE-BASED ITEM TRACKING

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in PROGRAM NAME WITH SPECIALIZATION NAME

by

NAME OF THE CANDIDATE (REG. NO.)

ADD ALL TEAM MEMBERS' NAMES AND REGISTER NUMBERS



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled “SmartPack: Closure-Verified RFID Smart Bag with Toggle-Based Item Tracking” submitted by NAME (REGISTER NO), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Guide Name.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: Signature of the Candidate

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled **"SmartPack: Closure-Verified RFID Smart Bag with Toggle-Based Item Tracking"** is prepared and submitted by Name (Reg No) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in** PROGRAM NAME is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Faculty:
Name: Dr./Prof. Guide Name
Date:

ABSTRACT

Forgetting essential items such as a wallet, keys, a charger or documents when leaving home is a common everyday problem, and existing item trackers only help to locate an object after the owner has noticed that it is missing. This project presents SmartPack, a low-cost smart bag built around an ESP32 microcontroller that verifies the contents of each compartment at the moment the compartment is closed. The prototype has two compartments, each with its own MFRC522 13.56 MHz RFID reader and magnetic reed switch, and the design extends to more compartments by adding one reader, one reed switch and one wiring-table row for each. Every item carries a passive RFID tag.

Each scan of a tagged item at a compartment's reader represents one crossing of that compartment's opening: the first scan marks the item as inside, the next scan marks it as taken out, and so on, so the firmware keeps an inside/outside state for every item through a simple toggle. When the reed switch reports that the compartment has been closed, the firmware checks that every item assigned to that compartment is marked inside. If any is not, a red LED and a buzzer alert the user immediately and the identities of the missing items are sent over Bluetooth Low Energy to a Flutter mobile application.

The application manages a bag with several compartments, adds an item by tapping its tag on the reader, shows the live in/out status of every item, and keeps item names on the phone so that the bag itself stores only anonymous tag identifiers. The firmware, the communication protocol and the application have been implemented; the core logic and the protocol parser are covered by automated tests, the firmware compiles for the ESP32, and a structured plan for hardware bench trials is defined as the next stage of the work.

Keywords: RFID, ESP32, MFRC522, reed switch, Bluetooth Low Energy, Flutter, smart bag, item verification.

TABLE OF CONTENTS

LIST OF FIGURES ..... **vi**
LIST OF TABLES ..... **vii**
LIST OF ACRONYMS ..... **viii**
CHAPTER – 1 INTRODUCTION **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Research Gap 1
1.4 Objectives 2
1.5 Novelty and Contribution 2
1.6 Scope and Limitations 3
1.7 Organization of the Report 3
CHAPTER – 2 RELATED WORK **4**
2.1 RFID-Based Bag and Personal-Item Systems 4
2.2 Missing-Tag Detection 5
2.3 Mobile Applications in Related Work 5
2.4 Summary of the Gap 6
CHAPTER – 3 PROPOSED METHODOLOGY **7**
3.1 System Overview 7
3.2 Hardware Design 8
3.3 Toggle-Based Item Tracking 10
3.4 Closure-Triggered Verification 11
3.5 Local Feedback 13
3.6 Firmware Architecture 13
3.7 BLE Communication Protocol 14
3.8 Mobile Application 15
CHAPTER – 4 EXPERIMENTAL SETUP **18**
4.1 DATASET 18
4.2 EVALUATION METRICS 18
4.3 IMPLEMENTATION DETAILS 19
CHAPTER – 5 RESULTS & DISCUSSION **21**
5.1 Verification Completed So Far 21
5.2 Planned Trial Protocol and Result Tables 21
5.3 Discussion 23
CHAPTER – 6 CONCLUSION & FUTURE WORK **25**
6.1 Conclusion 25
6.2 Future Work 25

REFERENCES..... 26

LIST OF FIGURES

Figure 1: System architecture of SmartPack (prototype with two compartments).....7
Figure 2: Inside/outside state of a tracked item.....10
Figure 3: Closure verification flow.....12
Figure 4: Adding an item by tap-to-enrol.....16

LIST OF TABLES

Table 1: Summary of the surveyed works and their relation to SmartPack.....4
Table 2: Approaches to bag-content checking by trigger.....6
Table 3: Hardware components and their roles.....8
Table 4: Final GPIO assignment of the ESP32.....8
Table 5: Outcome of a tag scan.....11
Table 6: LED and buzzer patterns.....13
Table 7: Commands from the app to the bag.....14
Table 8: Events from the bag to the app.....15
Table 9: Evaluation metrics.....18
Table 10: Development environment.....19
Table 11: Verification status of the implementation.....21
Table 12: Expected behaviour in the basic closure scenarios.....22
Table 13: Toggle accuracy trial (to be completed).....22
Table 14: Closure verification trial (to be completed).....22
Table 15: Latency, range and power (to be completed).....22

LIST OF ACRONYMS

API	Application Programming Interface
APK	Android Package
BLE	Bluetooth Low Energy
GATT	Generic Attribute Profile
GPIO	General-Purpose Input/Output
HF	High Frequency (13.56 MHz)
IoT	Internet of Things
LED	Light-Emitting Diode
MCU	Microcontroller Unit
MTU	Maximum Transmission Unit
NFC	Near-Field Communication
NVS	Non-Volatile Storage
RFID	Radio-Frequency Identification
RST	Reset (pin)
SPI	Serial Peripheral Interface
SS / CS	Slave Select / Chip Select
UHF	Ultra High Frequency
UI	User Interface
UID	Unique Identifier (of an RFID tag)
UUID	Universally Unique Identifier

CHAPTER – 1
INTRODUCTION

1.1 Background

People routinely leave home, office or a hostel carrying a bag that should contain a few essential things: a wallet, house keys, a phone charger, an identity card, a laptop or a notebook. Forgetting one of them costs a return trip, a missed meeting or an anxious day. The usual remedy is a mental checklist, which fails exactly when the person is hurried or distracted.

Three technologies are now cheap enough to automate that checklist inside the bag itself. Passive RFID tags cost very little, need no battery and carry a unique identifier. A microcontroller such as the ESP32 can read tags, run decision logic and talk to a phone over Bluetooth Low Energy (BLE) from a single low-cost board. A magnetic reed switch, a sealed contact that closes in the presence of a magnet, gives a reliable, essentially free way to know when a flap or zip has been closed.

SmartPack combines these three parts into a bag that answers one question at the moment it matters: when a compartment is closed, is everything that is supposed to be inside actually inside?

1.2 Problem Statement

People often forget essential items (chargers, wallets, keys, documents) when leaving home, the office or on a trip, because there is no easy way to check a bag's contents before stepping out. This leads to wasted trips, missed work and daily stress. The need is for an inexpensive, personal, bag-level system that checks the contents automatically and alerts the user before they leave.

1.3 Research Gap

Bluetooth item trackers such as Tile and AirTag only help to find a lost item after the owner notices it is missing; they do not check a bag before the owner leaves. RFID inventory systems, on the other hand, are designed for warehouses, airports and logistics rather than for one person managing a few personal bags [17], [2], [10]. Research prototypes of RFID smart bags exist [16], [5], [1], [13], but they typically keep the reader powered continuously or scan when an item is placed, and none of the works reviewed uses the act of closing a compartment as the event that triggers verification.

1.4 Objectives

- Design an ESP32-based bag with an independent RFID reader and reed switch for each compartment (two in the prototype), in a way that extends to more compartments.
- Treat every RFID scan as one crossing of a compartment's opening, so that the first scan of an item means **in** and the next scan means **out**.
- Verify, at the moment a compartment is closed, that every item assigned to that compartment is inside, and give immediate local feedback through LEDs and a buzzer.
- Report the exact missing items to a mobile application over BLE, without the bag ever storing item names.
- Build a mobile application that lets the user add items to each compartment by tapping their tags, rename compartments, view live status and receive missing-item alerts.
- Define an evaluation method (accuracy, false alarms, latency, power) for the finished prototype.

1.5 Novelty and Contribution

- **Closure-triggered verification.** The check happens when the reed switch reports that a compartment has been closed, not continuously and not when an item is placed.
- **Crossing-based toggle tracking.** Instead of asking a reader to "see" every tag inside the bag, each scan is interpreted as one crossing of the compartment opening, which gives a simple inside/outside state per item.
- **Per-compartment sensing.** Each compartment has its own reader and reed switch, so a bag with several compartments is verified compartment by compartment. The prototype has two compartments; the number is a build setting, so the same design extends to more.
- **Hardware/software split for privacy.** The bag stores only tag identifiers; names live on the phone, so a lost bag does not reveal what its owner carries.
- **Tap-to-enrol workflow.** An item is added to a compartment by naming it in the app and tapping its tag on the reader.

1.5.1 Evolution of the design

The initial concept (ZipCheck) used a single reader that scanned all tags for two to three seconds after a zip closure. During implementation the design was refined to the version reported here: two readers (one per compartment) and a scan-toggle model in which each scan represents an item crossing the compartment opening, with the reed switch

triggering the verification of the resulting inside/outside state. Section 5.3 discusses the consequences of this choice.

1.6 Scope and Limitations

The prototype is built with two compartments and supports up to sixteen tagged items per compartment, a single phone connection at a time, and Android and iOS applications built from one Flutter code base. The number of compartments is a build setting rather than a fixed property of the design (Section 3.2.4), but configurations with more than two compartments have not been built or tested on hardware. The report covers the hardware design, the firmware, the BLE protocol and the application. Quantitative field results are not reported because the hardware trials are still to be carried out; Chapter 5 states precisely what has been verified so far and what remains.

1.7 Organization of the Report

Chapter 2 reviews related work. Chapter 3 presents the proposed methodology: architecture, hardware, toggle tracking, closure verification, firmware, BLE protocol and the mobile application. Chapter 4 describes the experimental setup, evaluation metrics and implementation details. Chapter 5 reports what has been verified, the planned trial protocol and a discussion of design trade-offs. Chapter 6 concludes and lists future work.

CHAPTER – 2
RELATED WORK

2.1 RFID-Based Bag and Personal-Item Systems

Several works place an RFID reader in a bag and compare the detected tags with a saved list. Sarkar et al. [16] use passive tags and a reader inside the bag to track presence and raise alerts, relying on continuous or periodic scanning. Chauhan and Nigam [5] combine an RFID reader, a load cell and a NodeMCU to identify and count items against a saved school timetable. Das et al. [6] describe an Arduino-based bag with RFID detection, GPS tracking and solar charging, and Varma et al. [18] use a Raspberry Pi with GPS and a geo-fencing layer. Antad et al. [1] scan items as they are placed in a bag and check them against a stored database.

The closest ideas to the toggle model used here are the interactive log bag of Mohd Yazed and Mahmud [13], which logs tagged items as they go in and out of the bag and reminds the user of forgotten items with an LED strip, and the IPURSE system of Watfa et al. [19], which tracks purse contents and alerts the user when something is missing. SmartPack differs by using one reader and one reed switch per compartment, by tying verification to compartment closure, and by managing items from a phone application.

Outside the bag, Hsu et al. [12] detect the objects a person takes through a doorway and remind them of items usually carried, and Calo et al. [4] use a UHF base device and a phone application to notify the user of forgotten items. Hallberg et al. [11] locate misplaced RFID-tagged items inside a home by comparing signal strengths with reference tags. Large-scale systems for airports and logistics [17], [2] show that RFID is a mature identification technology but target networked, multi-checkpoint settings rather than one personal bag.

Table 1: Summary of the surveyed works and their relation to SmartPack

Ref.	Work (year)	Approach	Difference from SmartPack
[16]	Smart bag tracking and alert system using RFID (2017)	Passive tags; reader inside the bag; alert on missing item	Continuous / periodic scanning, not closure-triggered
[5]	Smart bag based on RFID and IoT (2021)	RFID + load cell + NodeMCU vs. saved schedule	Always-on; schedule-based rather than closure-triggered
[6]	Smart bag, LugTrack (2019)	Arduino, RFID, GPS, solar charging	Broader tracking scope; no closure trigger
[2]	IoT RFID luggage sorting and tracking (2024)	RFID + IoT for automatic luggage sorting	Airport logistics, not a personal bag
[17]	Baggage tracing for airports using RFID and IoT (2016)	Cloud-connected tag generation and tracking	Multi-checkpoint network scenario

Ref.	Work (year)	Approach	Difference from SmartPack
[18]	Multifunctional bag monitoring system (2022)	Raspberry Pi, GPS, RFID, geo-fencing	Heavier platform than a microcontroller
[1]	IoT smart bag with security system and rainwear (2023)	Scans items on placement; database check; alert	Triggered by placement, not by closure
[4]	Forgotten-item detection using UHF RFID (2018)	Base device + phone app notification	UHF, vehicle / equipment setting
[11]	Localisation of forgotten items using RFID (2009)	Signal strength vs. reference tags in a home	Locates items; no present/absent checklist
[13]	RFID-based interactive log bag (2016)	Logs tagged items going in and out; LED reminder	No per-compartment closure verification
[12]	RFID-based reminder system for smart home (2011)	Doorway reader; reminder list from history and calendar	Fixed doorway reader, not in the bag
[19]	Intelligent RFID system for purses / handbags, IPURSE (2011)	RFID + NFC; alerts for missing items	Periodic or manual scanning
[10]	High time-efficient missing tag detection (2022)	Multi-reader missing-tag detection protocol	Warehouse scale; algorithmic focus

2.2 Missing-Tag Detection

The decision at the heart of SmartPack is a missing-set computation: which expected tags are not accounted for. Guo et al. [10] study this problem for large integrated RFID systems with several readers and heterogeneous tags and propose a protocol that improves detection time. That work addresses warehouse-scale populations and radio-level collision handling; for a bag with a few dozen tags, a direct set comparison on the microcontroller is sufficient, which keeps SmartPack independent of the network and of any cloud service.

2.3 Mobile Applications in Related Work

Only some of the surveyed works include a phone application. The log bag of Mohd Yazed and Mahmud [13] saves its in/out log on an SD card and shows it in an Android application; Calo et al. [4] pair a base device with a companion mobile application that notifies the user when an item is missing; and IPURSE [19] is built on a mobile platform that alerts the user and takes context such as the calendar and weather into account. Chauhan and Nigam [5] report alerts and location sharing. For the remaining works, the descriptions available in this survey mention local alerts, tracking or logistics features rather than an application for managing the items of a bag.

SmartPack's application is concerned with managing the bag rather than only displaying alerts. It keeps the item library and names on the phone, adds items by tap-to-

enrol, supports several compartments, shows the live in/out state of every item, synchronises with the bag at each connection, lets the user rename compartments, correct an item’s state and delete items, and names the missing items when a compartment is closed.

2.4 Summary of the Gap

Table 2 groups the surveyed approaches by what triggers the check. Most keep the reader on continuously or scan when an item is placed, and several target airports or warehouses. None of the surveyed systems, as described in the sources reviewed, uses the closing of a compartment as the trigger for verification, and none is described as combining per-compartment readers with a crossing-based in/out model managed from a phone application. This comparison rests on the abstracts and descriptions of the works and should be rechecked against their full texts.

Table 2: Approaches to bag-content checking by trigger

Trigger	Works	Limitation for a personal bag
Continuous or periodic scanning	[16], [5], [18], [19]	Reader powered constantly; shorter battery life
Scan on item placement	[1], [13]	Checks a single placement, not the final state at closure
Fixed doorway reader	[12], [4]	Only works at one location; not carried with the bag
Warehouse / airport infrastructure	[17], [2], [10]	Networked infrastructure; not for one person
Compartment closure (this work)	SmartPack	Check happens exactly when the compartment is closed

CHAPTER – 3
PROPOSED METHODOLOGY

3.1 System Overview

SmartPack has three parts, shown in Figure 1. The **bag hardware** senses (two RFID readers, two reed switches), decides (the ESP32) and gives local feedback (two LEDs and a buzzer). The **firmware** on the ESP32 turns raw scans and reed-switch levels into item states and closure checks, stores the configuration in flash and speaks a small text protocol over BLE. The **mobile application** keeps the item names, lets the user add and remove items, shows the live status and displays missing-item alerts.

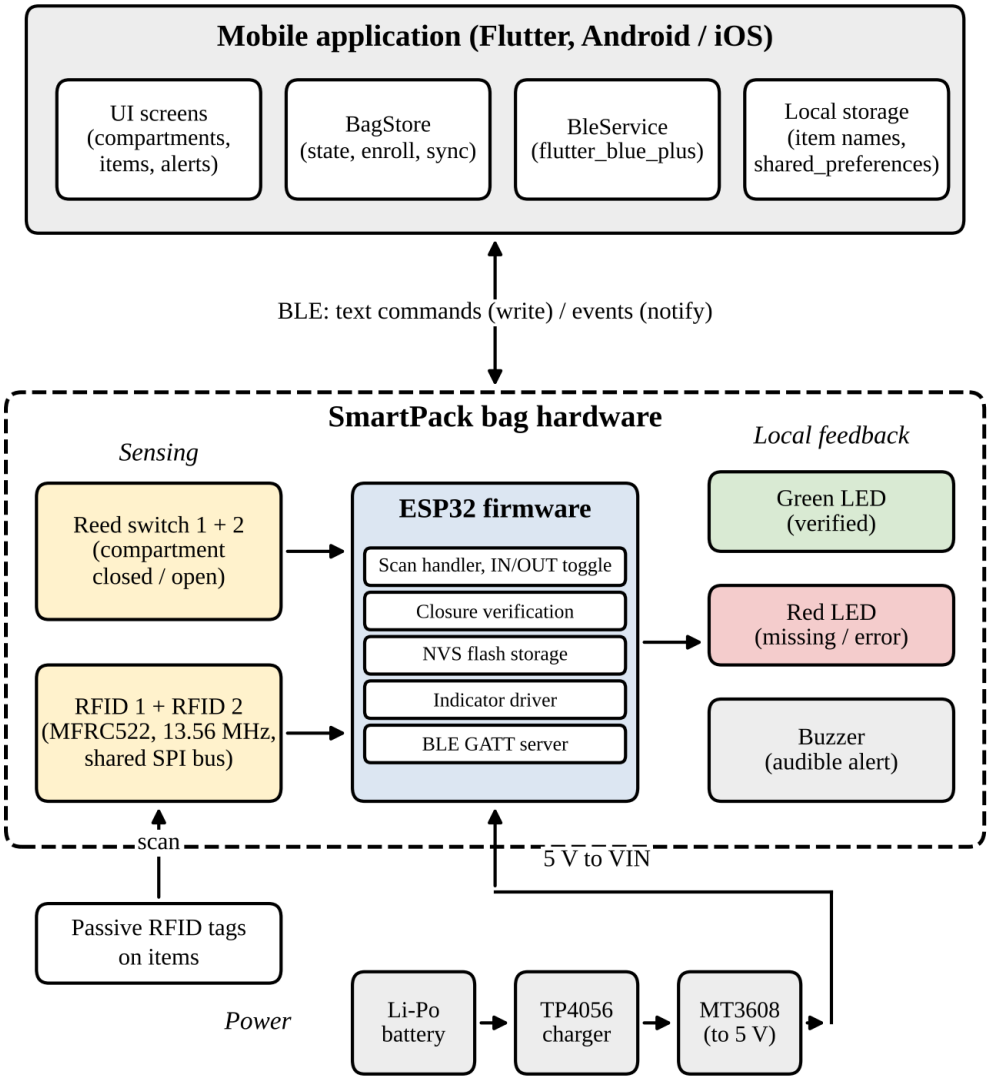


Figure 1: System architecture of SmartPack (prototype with two compartments)

The division of work is deliberate. Time-critical operations (reading tags, debouncing the reed switch, lighting the LEDs) happen on the bag and work with no phone nearby.

The phone is needed only to configure the bag and to receive detailed alerts, and the bag never needs to know what an item is called.

3.2 Hardware Design

3.2.1 Components

Table 3: Hardware components and their roles

Component	Role in SmartPack
ESP32 (ESP32-WROOM-32 development board)	Runs the firmware; SPI master for both readers; reads reed switches; drives LEDs and buzzer; BLE server [7]
2 × MFRC522 RFID reader (13.56 MHz)	One per compartment; reads the UID of passive tags over SPI [14]
Passive 13.56 MHz RFID tags	One per tracked item; no battery; unique identifier (UID)
2 × reed switch + magnet	One per compartment; closed contact means the compartment is closed
Green LED and red LED (220–330 Ω resistors)	Green: verified / item crossed; red: missing item, unknown tag or fault
Active buzzer	Audible confirmation and warning patterns
Li-Po battery, TP4056, MT3608	Rechargeable power: charger/protection, then boost to about 5 V for the ESP32

3.2.2 Pin Mapping

Both readers share the SPI bus (SCK, MISO, MOSI) and have separate slave-select and reset pins, so the two readers can be addressed independently. Each reed switch is wired between a GPIO and ground and read with the internal pull-up enabled, so a closed compartment reads LOW and an open one reads HIGH. Table 4 lists the final assignment.

Table 4: Final GPIO assignment of the ESP32

Component	Signal	ESP32 GPIO	Note
RFID 1 (compartment 1)	SDA / SS / CS	5	Unique to reader 1
RFID 1 (compartment 1)	RST	21	Unique to reader 1
RFID 2 (compartment 2)	SDA / SS / CS	16	Unique to reader 2
RFID 2 (compartment 2)	RST	17	Unique to reader 2
Shared SPI bus	SCK	18	Both readers
Shared SPI bus	MISO	19	Both readers

Component	Signal	ESP32 GPIO	Note
Shared SPI bus	MOSI	23	Both readers
Reed switch 1	Signal	27	Other wire to GND; CLOSED = LOW
Reed switch 2	Signal	26	Other wire to GND; CLOSED = LOW
Green LED	Anode (+)	25	Through 220–330 Ω
Red LED	Anode (+)	33	Through 220–330 Ω
Buzzer	Positive (+)	32	Small low-current active buzzer
MFRC522 VCC (both)	Supply	3V3	Never 5 V, VIN or the battery directly

3.2.3 Power

The Li-Po battery charges through a TP4056 module. An MT3608 boost converter, adjusted with a multimeter to approximately 5.0 V before the ESP32 is connected, feeds the ESP32 VIN pin. The MFRC522 modules are powered only from the ESP32's 3.3 V pin, which is a hard safety rule because the readers are damaged by 5 V. All grounds are common.

3.2.4 Scaling to More Compartments

The prototype uses two compartments, but nothing in the design is specific to the number two. Everything that belongs to a compartment (its reader, its reed switch, its item list and its closure check) is addressed by a compartment index, and the number of compartments is a single build setting. Adding a compartment takes four steps:

- **Hardware.** Connect another MFRC522 reader to the shared SPI bus with its own slave-select and reset pins, and another reed switch to a free GPIO.
- **Wiring table.** Add one row (slave select, reset, reed switch) for the new compartment to the firmware's wiring table.
- **Compartment count.** Raise the compartment-count setting in the firmware and the matching constant in the application. A compile-time check stops the firmware build if the table and the count disagree.
- **Nothing else.** The BLE protocol numbers compartments from 1 to N, the command parser accepts any valid compartment number, and the storage layout and the application's compartment cards are generated from the count.

Two practical limits apply. Each compartment needs three GPIOs (slave select, reset and the reed switch), so the number of compartments is bounded by the free pins of the board; the ESP32's input-only pins (GPIO 34 to 39) have no internal pull-up and would need an external resistor if used for reed switches. In addition, the readers are polled in

turn, so the time between two polls of the same reader grows with the number of compartments (60 ms multiplied by the count in the present firmware). Port expanders or a multiplexer for the slave-select lines would be needed for many compartments. The pin-configuration document describes the two-compartment prototype and is unchanged.

3.3 Toggle-Based Item Tracking

A tag read by a compartment's reader means that the item is passing through that compartment's opening. Because the reader cannot tell the direction of travel, SmartPack uses the order of events instead: the first scan puts the item inside, the next scan takes it out, and so on. Each item therefore has one bit of state, INSIDE or OUTSIDE, as shown in Figure 2. A newly added item starts OUTSIDE, so the first scan puts it in.

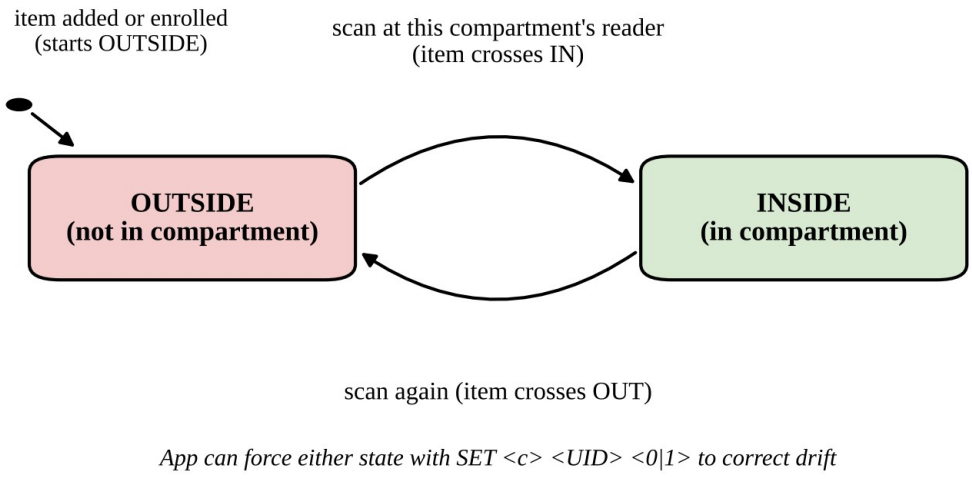


Figure 2: Inside/outside state of a tracked item

Several rules keep the toggle reliable in practice:

- **One reading per presentation.** After a tag is read, the reader sends it a halt command, so the tag must leave the field before it can be read again. In addition, the same UID on the same reader is ignored for 1.5 s.
- **Alternating readers.** The firmware polls the readers in turn every 60 ms, so with two compartments each reader is polled about every 120 ms, and the shared SPI bus is never used by two readers at once.
- **Scans count only while the compartment is open.** A tag scanned at a closed compartment is reported as ignored, because items cross the opening only when it is open. This is a single configuration constant in the firmware.

- **Unknown tags are reported, not toggled.** A tag that is not assigned to the compartment produces a short red flash and beep and an UNKNOWN event; it never changes any item's state.
- **Manual correction.** If the bag and the user ever disagree (for example a scan was missed), the app can force an item's state with a SET command.

Table 5: Outcome of a tag scan

Situation	Bag event sent to the app	Local feedback
Assigned item, state was OUTSIDE	IN c UID	One green flash and short beep
Assigned item, state was INSIDE	OUT c UID	Two green flashes and beeps
Tag not assigned to this compartment	UNKNOWN c UID	One short red flash and beep
Compartment is closed	IGNORED_CLOSED c UID	None
Same tag within 1.5 s	None (ignored)	None

3.4 Closure-Triggered Verification

Each reed switch is read continuously and debounced: a new level must be stable for 60 ms before it is accepted. On an accepted OPEN to CLOSED change the firmware verifies that compartment. Let A_c be the set of items assigned to compartment c and s_i the state of item i (INSIDE or OUTSIDE). The missing set is

$$M_c = \{ i \in A_c : s_i = \text{OUTSIDE} \} \qquad (3.1)$$

If A_c is empty there is nothing to verify and no alert is raised. If M_c is empty the green LED lights for 1.5 s with a short beep. Otherwise the red LED lights for about 2.8 s with three beeps, one MISS line is sent for each missing item and a final VERIFY line gives the count. Figure 3 shows the complete decision flow.

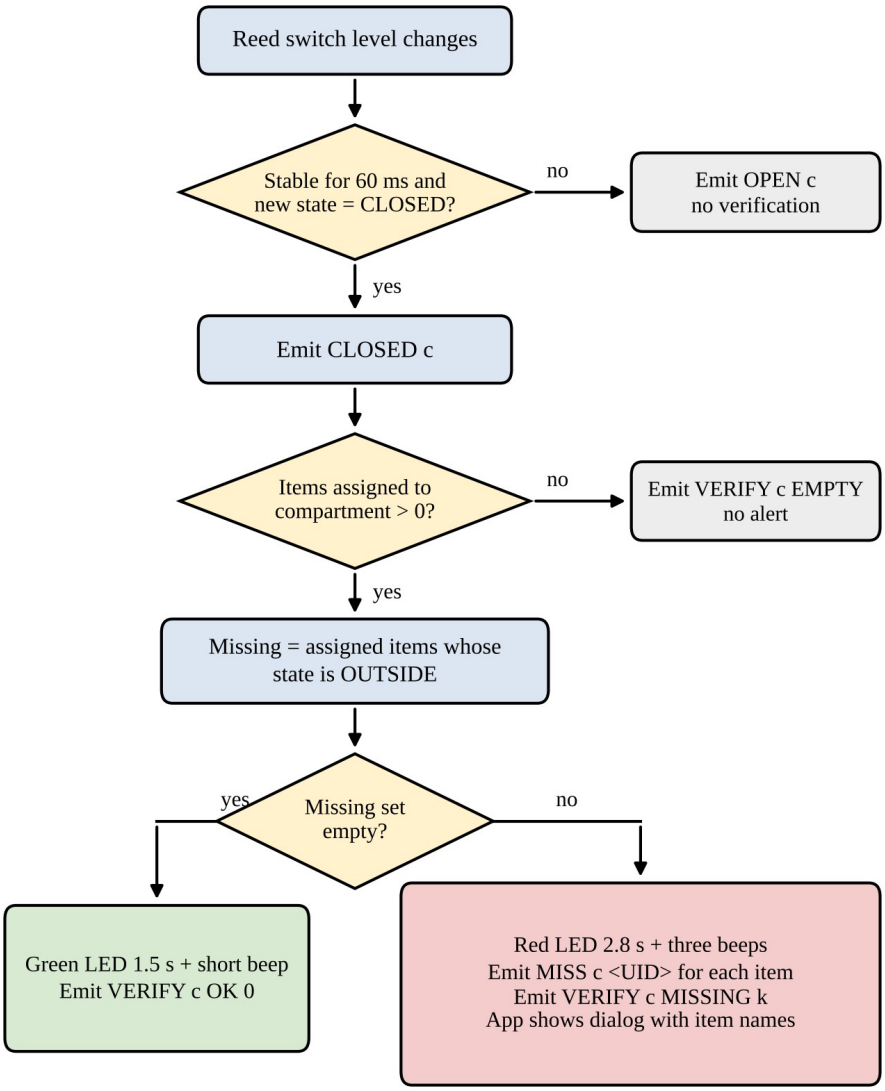


Figure 3: Closure verification flow

On start-up the firmware adopts the current reed-switch states silently, so powering the bag on with a compartment already closed does not raise a false alarm.

The following pseudocode summarises the main loop of the firmware.

```
setup():
  load saved configuration from flash
  reset both readers; read MFRC522 version (fault if 0x00 / 0xFF)
  adopt reed states silently; start BLE server
loop():
  handle commands from BLE and serial
  for each compartment c:
    debounce reed(c)
    on stable CLOSED edge: verify(c)
  every 60 ms, poll the next reader r:
    if a tag UID is read and it is not a repeat within 1.5 s:
      if enrolling:     add UID to the requested compartment
      elif r is closed: report IGNORED_CLOSED
      else:             toggle state, save, report IN/OUT/UNKNOWN
```

```
enrol timeout (30 s); update LED/buzzer; send queued BLE events
```

3.5 Local Feedback

The LEDs and buzzer give the user immediate feedback that does not depend on the phone. The patterns are generated by a small non-blocking driver, so the main loop never waits for a pattern to finish and scanning continues while a pattern plays.

Table 6: LED and buzzer patterns

Event	LED	Buzzer
Item crossed IN	Green, one 80 ms flash	One short beep
Item crossed OUT	Green, two 80 ms flashes	Two short beeps
Compartment verified	Green, 1.5 s	One 150 ms beep
Items missing at closure	Red, about 2.8 s	Three 200 ms beeps
Unknown tag	Red, one 60 ms flash	One 60 ms beep
Tag enrolled	Green, three 70 ms flashes	Three 70 ms beeps
Reader not responding at start-up	Red, two 500 ms flashes	Two 500 ms beeps

3.6 Firmware Architecture

The firmware is written for the Arduino framework on the ESP32. The MFRC522 readers are driven through the MFRC522v2 library [15], and the BLE server uses the ESP32 BLE library. The code is split so that the decision logic is independent of the hardware:

- **Core logic** (a hardware-independent header): the tag identifier type, the item list and state of each compartment, the toggle, the closure check and the command parser. Because it has no Arduino dependency it can be unit-tested on a PC.
- **Main loop**: reed debouncing, reader polling, enrolment mode and command dispatch, all driven by a wiring table with one row per compartment.
- **Indicator driver**: the non-blocking LED and buzzer patterns of the table above.
- **Storage**: the whole configuration and all item states are written to the ESP32’s non-volatile storage as a single versioned blob after every change, so the bag keeps its items and their states across a power cycle.
- **BLE link**: a GATT service with one write characteristic for commands and one notify characteristic for events. BLE callbacks run in a separate task, so incoming commands are placed in a queue and executed by the main loop; outgoing events are queued and sent 25 ms apart so that bursts such as a list of missing items are not lost.

- **Serial console**: the same text commands are accepted over USB, which allows the whole system to be tested without the app.

3.7 BLE Communication Protocol

The bag advertises as “SmartPack” with a custom service, requests a 185-byte MTU, and exchanges one short text line per message. A text protocol was chosen because it is easy to read in a serial monitor and to generate and parse in the app. The service UUID is 6e5a0001-5b1c-4f6e-9a3d-2f7d4c0b5a10; commands are written to the characteristic ending 0002 and events arrive as notifications on the characteristic ending 0003 [3]. Compartments are numbered 1 to N on the wire (1 and 2 in the prototype) and UIDs are upper-case hexadecimal.

Table 7: Commands from the app to the bag

Command	Effect
ENROLL c	The next tag scanned on either reader is added to compartment c (30 s timeout)
CANCEL	Leave enrol mode
ADD c UID	Assign a tag to compartment c
DEL c UID	Remove a tag from compartment c
CLEAR c	Remove every item of compartment c
SET c UID 0|1	Force an item’s state (0 outside, 1 inside)
LIST c	Reply with one ITEM line per item of compartment c
STATUS	Reply with one STATE line per compartment

Table 8: Events from the bag to the app

Event	Meaning
READY	Bag has started
IN c UID / OUT c UID	An item crossed into / out of compartment c
UNKNOWN c UID / IGNORED_CLOSED c UID	Tag not assigned / scanned while the compartment was closed
OPEN c / CLOSED c	Debounced reed-switch change
MISS c UID … VERIFY c MISSING k	At closure: each missing item, then the count
VERIFY c OK 0 / VERIFY c EMPTY 0	At closure: all present / nothing assigned
ENROLLED c UID, ENROLL_DUP c UID, ENROLL_TIMEOUT	Results of an enrol request
ITEM c UID s, STATE c OPEN| CLOSED n m	Replies to LIST and STATUS
OK cmd, ERR reason	Command accepted / rejected

3.8 Mobile Application

3.8.1 Architecture

The application is written in Flutter [9], so one code base builds for Android and iOS. It is organised in four layers, shown inside the application block of Figure 1:

- **Protocol layer**: a pure-Dart parser that turns each event line from the bag into a typed event object. It has no Flutter or Bluetooth dependency and is unit-tested.
- **BLE service**: scanning for the SmartPack service, connecting, requesting a larger MTU, subscribing to the event characteristic and writing commands, built on the flutter_blue_plus package [8].
- **State layer (BagStore)**: holds the compartments and their items, handles every event, runs the enrol flow, synchronises with the bag on connection and raises the missing-item alert. It notifies the UI through the Provider package.
- **UI and local storage**: the home screen with one card per compartment, and a local JSON store (shared preferences) that keeps compartment names, item names, UIDs and states on the phone.

3.8.2 Data Model

A bag has a fixed number of compartments, set by one constant that matches the firmware (two in this prototype). Each compartment has a name and a list of items; each item has a UID, a human-readable name and an inside flag. The bag itself knows only

the UIDs and the inside flags. This split means that a lost or stolen bag cannot reveal what its owner carries.

3.8.3 Adding Compartments and Items

Items are added to a compartment by tapping their tag on the reader. The user opens a compartment's card, taps **Add item**, types a name such as "Wallet" and then taps the tag on that bag's reader. The app sends ENROLL, the bag replies ENROLLED with the tag's UID, and the app stores the name and UID together. The new item starts as outside; its first scan puts it inside. Figure 4 shows the sequence. A compartment can be renamed by tapping its title. An item is deleted with the bin icon on its row, after a confirmation dialog; the app then sends a DEL command to the bag and removes the item from the phone. Deletion is allowed only while the app is connected to the bag, so that the two never disagree about which tags are assigned.

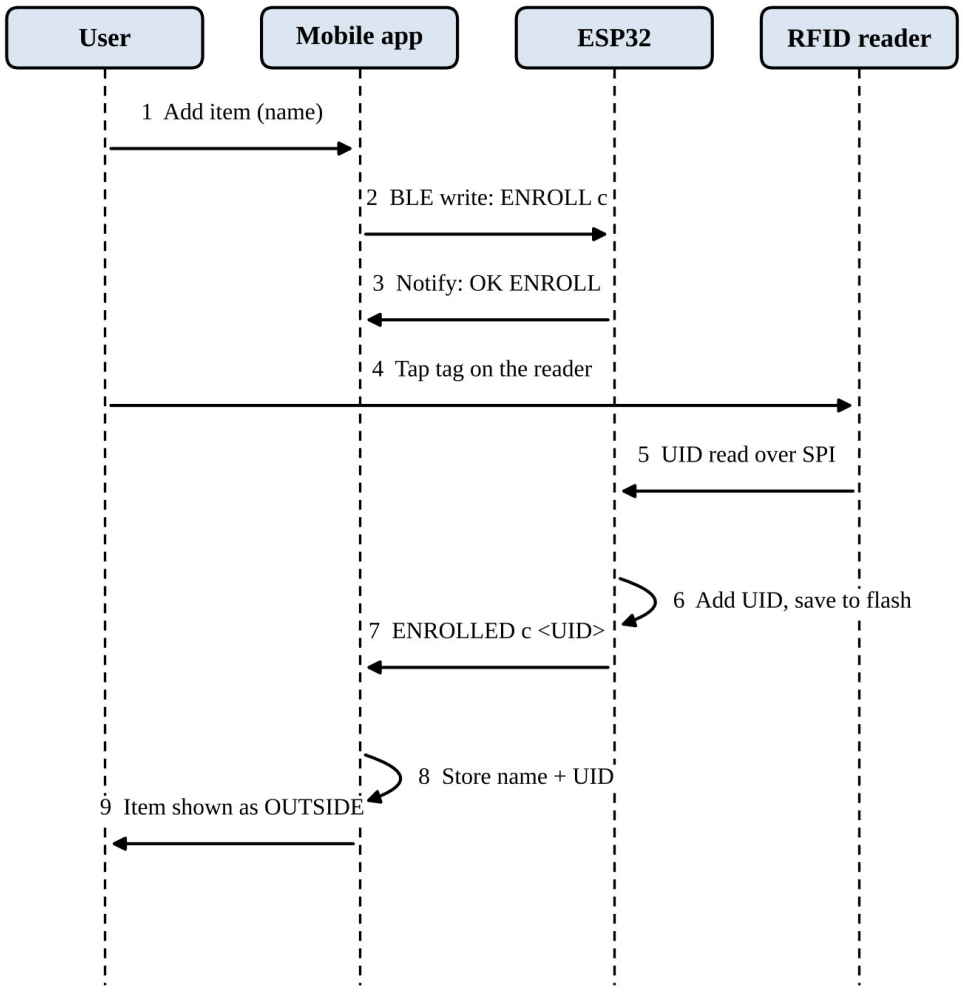


Figure 4: Adding an item by tap-to-enrol

3.8.4 Synchronisation and Alerts

When the app connects it asks the bag for each compartment’s item list. Items that the phone knows but the bag has lost are sent again with ADD; the in/out state of every item is taken from the bag; and any tag that the bag knows but the phone has no name for appears as “Unnamed tag XXXX” so that it can be renamed. When a closure check finds missing items the app maps each MISS UID to its name, shows a dialog such as “Missing in Compartment 1: Wallet” and vibrates the phone. The user can also override an item’s state with the switch on its row, which sends a SET command.

CHAPTER – 4
EXPERIMENTAL SETUP

4.1 DATASET

SmartPack does not learn from data: its decisions are deterministic set and state operations, so there is no training dataset. The data used for evaluation is a **test set of tagged items and trial scripts**:

- A set of passive 13.56 MHz RFID tags attached to representative items (for example wallet, keys, charger, identity card, notebook, bottle). Record the tag type and the actual items used
- Two compartments of one bag, each with its own reader and reed switch, fitted as described in Chapter 3.
- Scripted trial sequences: put items in one by one, remove one, close the compartment, repeat; each sequence has a known expected outcome (Section 5.2).

Tag identifiers are read by the system itself, so no external data collection is required.

4.2 EVALUATION METRICS

The metrics below cover the two decisions the system makes (toggling an item's state, and verifying a compartment at closure) and the practical qualities of the device.

Table 9: Evaluation metrics

Metric	Definition	Collected from
Toggle accuracy	Scans that produced the expected IN/OUT change ÷ total deliberate scans	Serial/BLE event log
Missed-scan rate	Deliberate presentations of a tag with no event ÷ total presentations	Event log
Double-count rate	Single presentations that produced two state changes ÷ total presentations	Event log
Verification correctness	Closures where the alert (green / red) matched the true contents ÷ total closures	Trial sheet + log
False-alarm rate	Closures with all items truly inside but a missing alert ÷ such closures	Trial sheet + log
Missed-detection rate	Closures with an item truly missing but no alert ÷ such closures	Trial sheet + log
Closure-to-alert latency	Time from reed closing to LED on	Video or logic analyser
Read range / orientation	Largest distance and tag angles at which a tag is reliably read	Ruler, repeated scans
Battery run time	Hours of operation from a charged Li-Po cell in always-on mode	Timed discharge test

4.3 IMPLEMENTATION DETAILS

4.3.1 Software Toolchain

Table 10: Development environment

Item	Version / configuration
Firmware framework	Arduino framework for ESP32 (Arduino IDE; the same sources also build with PlatformIO), board “ESP32 Dev Module”
RFID library	MFRC522v2 (Arduino_MFRC522v2) [15]
BLE and storage	ESP32 BLE library; Preferences (non-volatile storage)
Mobile application	Flutter 3.38.9 (stable), Dart 3.10.8 [9]
Application packages	flutter_blue_plus ^1.35.0 [8]; provider ^6.1.2; shared_preferences ^2.2.3
Android build tools	Android SDK 36.1.0, NDK 28.2.13676358, JDK 21 (bundled with Android Studio)
Source control	Git repository containing the firmware, the application, the protocol document and the pin configuration

4.3.2 Code Organisation

The repository is organised so that each concern lives in one place: the firmware sources and the hardware-independent core logic, a protocol document that both sides follow, the Flutter application, an Arduino IDE copy of the sketch, and the unchanged pin-configuration document. Key limits are set as constants in the firmware: the number of compartments (two in the prototype, a build setting), sixteen items per compartment, a 60 ms reed debounce, a 60 ms reader polling interval, a 1.5 s same-tag cooldown and a 30 s enrol timeout.

4.3.3 Testing Approach

- **Core logic tests on a PC.** The hardware-independent core is compiled with a normal C++ compiler and exercised by assertions covering hexadecimal UID conversion, duplicate assignment, per-compartment independence, the IN/OUT toggle, the missing-set computation, capacity limits, state forcing and the command parser, including malformed commands.
- **Protocol parser tests in Dart.** Event lines of every kind, and malformed lines, are fed to the application’s parser.
- **Compile checks.** The firmware is built for the ESP32 and the application is analysed and built with the Flutter toolchain.

- **Bench tests (planned).** The same text commands work over the USB serial console, so the toggle, enrol and closure behaviour can be tested with real tags and reed switches before the application is involved.

CHAPTER – 5
RESULTS & DISCUSSION

This chapter separates clearly what has been **verified so far** from what is **planned**. Hardware bench trials had not been completed when this report was written, so no accuracy, latency or battery figures are claimed; the result tables in Section 5.2 are prepared for those measurements.

5.1 Verification Completed So Far

Table 11: Verification status of the implementation

Item	Method	Outcome
Core logic (toggle, missing set, parser)	PC unit tests with assertions	All tests passed
Firmware build	Compiled for ESP32 Dev Module in Arduino IDE	Succeeded: 1,137,279 bytes of program storage (86% of 1,310,720); 44,912 bytes of dynamic memory (13% of 327,680). Measured before the wiring table was introduced; re-measure
Configurable compartment count	Core tests compiled for 2, 3 and 4 compartments; firmware sources type-checked for 2 and 3 compartments (types and syntax only, with stand-in Arduino headers); build stops if the wiring table has the wrong number of rows	Passed; not tried on hardware with more than two compartments
Application protocol parser	Four Dart unit tests (tag, reed/verify, replies, malformed input)	All four passed
Application build and install	Flutter analyse / release build for Android	Record outcome
Firmware on hardware	Upload to board; serial bench test	Record outcome
End-to-end app + bag	BLE connection, enrol, IN/OUT, closure alert	Record outcome

The firmware, including BLE and Wi-Fi support libraries, fits the default application partition of the ESP32 with 14% of program storage to spare, and uses 13% of dynamic memory, so the design leaves room for further features.

5.2 Planned Trial Protocol and Result Tables

The trials follow the metrics of Chapter 4. Table 12 lists the behaviour the firmware is designed to produce in each basic scenario; the “Observed” column is to be completed from the bench test.

Table 12: Expected behaviour in the basic closure scenarios

Scenario	Designed behaviour	Observed	Pass / fail
All assigned items scanned IN, compartment closed	Green LED 1.5 s + short beep; VERIFY c OK 0		
One item not scanned IN, compartment closed	Red LED + three beeps; MISS c UID; VERIFY c MISSING 1		
Item scanned IN, then scanned OUT, closed	Item counted as missing		
Compartment with no assigned items closed	No alert; VERIFY c EMPTY 0		
Tag from another compartment scanned	UNKNOWN c UID; short red flash; state unchanged		
Tag scanned while compartment closed	IGNORED_CLOSED c UID; state unchanged		
Tag held on reader for several seconds	Exactly one IN or OUT event		
Tag presented to one reader with another compartment's reader nearby	Only the intended reader reports the tag (no cross-read)		
Power cycle with items inside	States restored from flash; no alarm on start		

Table 13: Toggle accuracy trial (to be completed)

Reader	Scans	Correct	Missed	Double	Accuracy
Reader 1					
Reader 2					

Table 14: Closure verification trial (to be completed)

Case	Closures	Correct alert	False alarm	Missed detection
All items inside				n/a
One item missing			n/a	
Two or more missing			n/a	

Table 15: Latency, range and power (to be completed)

Measurement	Result	Method
Closure-to-LED latency		Video / logic analyser
Reliable read distance (cm)		Ruler
Average current, always-on (mA)		Multimeter
Run time on one charge (h)		Timed discharge

5.3 Discussion

5.3.1 What the closure check does and does not prove

Verification at closure compares the **recorded** state of each item with the list of assigned items. The record is built from scans, so the check is only as good as the user's scanning. If an item is placed in the compartment without being scanned, it is reported missing (a false alarm). If an item is scanned in but then not actually placed, the check cannot tell (a missed detection). This is the price of the crossing-based model, which needs only a short-range reader at the opening instead of a reader that can see everything inside the bag. A confirmation scan of the compartment's interior at closure is the natural next improvement (Section 6.2).

5.3.2 Toggle drift

Because the state is a toggle, one missed or extra scan inverts an item's state until it is corrected. The design limits this in three ways: a halted tag cannot be read again until it leaves the field, the same tag is ignored for 1.5 s, and scans at a closed compartment are ignored. If drift still occurs, the user can flip the item's switch in the app. The trial plan measures the missed-scan and double-count rates so that these protections can be judged with data.

5.3.3 Always-on operation and power

The original concept kept the ESP32 in deep sleep and woke it on closure. Reading crossings and answering the app at any moment needs the processor, the readers and BLE to stay active, so the present firmware is always on. This costs battery life compared with the sleeping design, and the planned power measurement will quantify it. Reducing the cost with duty-cycled reader polling and light sleep, and waking on a reed-switch change, is listed as future work.

5.3.4 Radio and tag considerations

The MFRC522 works at 13.56 MHz with a short read range, which suits a reader placed at a compartment opening where items are brought close to it, but it also means that a tag must be presented near the reader. Metal and foil inside a bag can detune HF antennas, so tag placement and reader mounting need to be tested in the real bag. UHF systems [4] read at greater range but need more power and larger hardware.

5.3.5 Security and privacy

Keeping names on the phone protects the owner's privacy if the bag is lost, because the bag stores only anonymous UIDs. The BLE link in the present prototype, however, does not require pairing or encryption, so any nearby device could connect and send

commands. Bonding with an encrypted characteristic, and a physical button that must be pressed to allow enrolment, are required before real-world use.

5.3.6 Number of compartments

The prototype was built with two compartments. Because the compartment count is a build setting and the compartments are independent in the logic, the same firmware and application should extend to more compartments by following Section 3.2.4, and the core logic tests pass when compiled for two, three and four compartments. The hardware effects of more readers have not been measured: loading of the shared SPI bus, current draw, the longer polling interval of each reader and, above all, whether the antennas of neighbouring readers read each other’s tags when the compartments are close together. If cross-reads occur, the readers need separation or shielding; the trial plan therefore includes a cross-read test.

5.3.7 Application behaviour

Missing-item alerts are shown by the application while it is running and connected. Alerts when the application is in the background, and automatic reconnection when the phone comes back into range, have not yet been implemented.

CHAPTER – 6
CONCLUSION & FUTURE WORK

6.1 Conclusion

This project designed and implemented SmartPack, a smart bag that verifies its contents at the moment a compartment is closed. Each compartment has its own RFID reader and reed switch (two in the prototype, extendable through a wiring-table row and a build setting), an RFID scan is treated as one crossing of the compartment's opening so that every item toggles between inside and outside, and closing the compartment triggers a check that every assigned item is inside. A missing item produces an immediate local alert and a Bluetooth message that the Flutter application turns into a notice naming the item. Item names stay on the phone, and new items are added by naming them and tapping their tag on the reader.

The decision logic and the application's protocol parser are covered by automated tests, and the firmware compiles for the ESP32 using 86% of the default program partition. Hardware bench trials against the metrics defined in Chapter 4 are the remaining step, and the result tables of Chapter 5 are ready to receive them.

6.2 Future Work

- **Confirmation scan at closure** to detect items that were not scanned in or that were scanned but not placed, combining the crossing model with a physical presence check.
- **Power reduction**: duty-cycled polling, light sleep and reed-switch wake-up, followed by a battery-life measurement.
- **Background notifications** and automatic reconnection in the mobile application.
- **BLE security**: pairing, encrypted characteristics and a physical enrol button.
- **More compartments and bags**: hardware trials with more than two compartments, port expanders for many compartments, runtime discovery of the compartment count by the application, several bags per application, and assigning the same item to different bags.
- **Field trials** with real users to measure usefulness and the rate of scanning mistakes in everyday use.
- **Packaging**: a compact PCB and a tag attachment method suited to different items.

REFERENCES

Numbers in square brackets in the text refer to this list. Entries highlighted in yellow could not be verified online and must be completed from the literature survey before submission.

[1]. Antad, S. M., Kharare, C., Kharat, J., Khanvilkar, O., Kharat, A., & Kharat, K. (2023). An IoT-based smart bag with integrated security system and rainwear. Venue, volume, pages / DOI to be confirmed – paper could not be located online

[2]. Basak, A., Debnath, S., & Pal, S. (2024). Design and testing of an IoT enabled RFID based automatic luggage sorting and tracking system. Venue, volume, pages / DOI to be confirmed – paper could not be located online

[3]. Bluetooth Special Interest Group. (n.d.). *Bluetooth core specification*. https://www.bluetooth.com/specifications/specs/

[4]. Calo, A.-M. V. M., et al. (2018). Real-time detection and notification system of forgotten items using ultra high frequency RFID technology. Venue, full author list and DOI to be confirmed – paper could not be located online

[5]. Chauhan, A. S., & Nigam, D. (2021). Smart bag based on RFID and Internet of Things. In *Proceedings of ICTCon2021* (*AIJR Proceedings*, pp. 102–105). https://books.aijr.org/index.php/press/catalog/book/115/chapter/1127

[6]. Das, P., Sanyal, S., Sinha, A., Chakraborty, S., & Prabhakar, P. (2019). Smart bag (LugTrack). Venue, volume, pages / DOI to be confirmed – paper could not be located online

[7]. Espressif Systems. (n.d.). *ESP32 series datasheet*. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[8]. *flutter_blue_plus* [Computer software]. (n.d.). pub.dev. https://pub.dev/packages/flutter_blue_plus

[9]. Google. (n.d.). *Flutter documentation*. https://docs.flutter.dev

[10]. Guo, K., Xie, X., Qi, H., & Li, K. (2022). A high time-efficient missing tag detection scheme for integrated RFID systems. *Sensors, 22*(12), Article 4601. https://doi.org/10.3390/s22124601

[11]. Hallberg, J., Nugent, C., Davies, R., & Donnelly, M. (2009). Localisation of forgotten items using RFID technology. In *Proceedings of the 9th International Conference on Information Technology and Applications in Biomedicine (ITAB 2009)*. IEEE.

[12]. Hsu, H.-H., Lee, C.-N., & Chen, Y.-F. (2011). An RFID-based reminder system for smart home. In *Proceedings of the 25th IEEE International Conference on Advanced Information Networking and Applications (AINA 2011)* (pp. 264–269). IEEE.

[13]. Mohd Yazed, M. S., & Mahmud, F. (2016). RFID-based interactive log bag for poor memory prevention. *International Journal of Integrated Engineering*, *8*(1). https://journal.uthm.edu.my/index.php/ijie/article/view/1403

[14]. NXP Semiconductors. (n.d.). *MFRC522 standard performance MIFARE and NTAG frontend* [Product data sheet]. https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf

[15]. OSSLibraries. (n.d.). *Arduino_MFRC522v2* [Computer software]. GitHub. https://github.com/OSSLibraries/Arduino_MFRC522v2

[16]. Sarkar, S., Manna, S., & Datta, S. (2017). Smart bag tracking and alert system using RFID. Venue, volume, pages / DOI to be confirmed – paper could not be located online

[17]. Singh, A. K., Meshram, S., Gujar, T., & Wankhede, P. R. (2016). Baggage tracing and handling system using RFID and IoT for airports. Venue, volume, pages / DOI to be confirmed – paper could not be located online

[18]. Varma, R., Pavshe, P., Bhadane, A., & Pagare, S. (2022). Multifunctional bag monitoring system. *International Research Journal of Engineering and Technology (IRJET)*. Volume, issue, pages to be confirmed

[19]. Watfa, M. K., Kaur, M., & Daruwala, R. F. (2011). An intelligent RFID system. *International Journal of Intelligent Systems Technologies and Applications*, *10*(4), 377–395.